Week 10: Advanced SQL Query System

Scenario:

The Perfume company wants to perform advanced analysis on customer purchases, product pricing, and sales performance. The management team needs complex reports to identify high-value customers, expensive products, and business trends.

Students will implement Subqueries and Nested Queries to extract deeper insights from the Perfume Database.

Tasks to be Done:

1. Implement Subqueries and Nested Queries Concept:

A Subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

Types Covered:

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

2. Find Products Above Average Price Task:

- Find products priced above the average price.

```
SQL> SELECT BRAND_NAME,PRICE FROM BRAND WHERE PRICE>(SELECT AVG(PRICE) FROM BRAND);

BRAND_NAME                       PRICE
------------------------------ ----------
Versace                            2200
Calvin                             3500
YSL                                2500
Fogg                               2000
```

- Display product name, category, and price.

```
SQL> SELECT B.BRAND_NAME,C.CATEGORY_NAME,B.PRICE FROM BRAND B JOIN CATEGORY C ON B.CATEGORY_ID=C.CATEGORY_ID WHERE B.PRICE>(SELECT AVG(PRICE) FROM B
RAND);

BRAND_NAME                     CATEGORY_NAME                    PRICE
------------------------------ ------------------------------ ----------
Versace                        Oriental                         2200
Calvin                         Vanilla                          3500
YSL                            Aquatic                          2500
Fogg                           Luxury                           2000
```

- Identify expensive products in each category.

```
SQL> SELECT B.BRAND_NAME,B.CATEGORY_ID,B.PRICE FROM BRAND B WHERE B.PRICE=(SELECT MAX(B2.PRICE) FROM BRAND B2 WHERE B2.CATEGORY_ID=B.CATEGORY_ID);

BRAND_NAME                    CATEGORY_I      PRICE
----------------------------- ---------- ----------
Dior                          CAT-001          1500
Chanel                        CAT-002          1800
Gucci                         CAT-003          1200
Versace                       CAT-004          2200
Armani                        CAT-005          1400
Burberry                      CAT-006          1600
Calvin                        CAT-007          3500
YSL                           CAT-009          2500
Fogg                          CAT-010          2000
Hugo Boss                     CAT-008          1900

10 rows selected.
```

- Find products with price higher than a selected category average.

```
SQL> SELECT PRODUCT_ID, BRAND_NAME, PRICE, CATEGORY_ID FROM BRAND WHERE PRICE > (SELECT AVG(PRICE) FROM BRAND);

PRODUCT_ID BRAND_NAME                      PRICE CATEGORY_I
---------- ------------------------------ ---------- ----------
BR-04      Versace                          2200 CAT-004
BR-07      Calvin                           3500 CAT-007
BR-09      YSL                              2500 CAT-009
BR-10      Fogg                             2000 CAT-010
```

3. Identify Customers with Maximum Purchases Task:

- Customer who spent the highest amount.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID HAVING SUM(TOTAL_AMOUNT)=(SELECT MAX(SUM(TOTAL_AMOUNT)) FROM ORDERS GROUP
 BY CUSTOMER_ID);

CUSTOME SUM(TOTAL_AMOUNT)
------- -----------------
C-1005              4500
```

- Customers with maximum number of orders.

```
SQL> SELECT CUSTOMER_ID,COUNT(*) FROM ORDERS GROUP BY CUSTOMER_ID HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM ORDERS GROUP BY CUSTOMER_ID);

CUSTOME   COUNT(*)
------- ----------
C-1001           1
C-1004           1
C-1002           1
C-1003           1
C-1005           1
```

- Customers whose spending is above average.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID HAVING SUM(TOTAL_AMOUNT)>(SELECT AVG(TOTAL_AMOUNT) FROM ORDERS);

CUSTOME SUM(TOTAL_AMOUNT)
------- -----------------
C-1003              3500
C-1005              4500
```

- Top 5 valuable customers.

```
SQL> SELECT * FROM (SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID ORDER BY TOTAL_SPENDING DESC) WHERE ROWNUM<
=5;

CUSTOME TOTAL_SPENDING
------- --------------
C-1005            4500
C-1003            3500
C-1001            2500
C-1002            1800
C-1004            1200
```

4. Generate Complex Business Queries

Query 1: Best-Selling Product

Find products with maximum sales quantity. Tasks:

- Display product name.
- Total quantity sold.
- Total revenue generated.

```
SQL> SELECT B.BRAND_NAME,SUM(OD.QUANTITY) TOTAL_QUANTITY,SUM(OD.QUANTITY*B.PRICE) TOTAL_REVENUE FROM BRAND B JOIN ORDER_DETAILS OD ON B.PRODUCT_ID=O
D.PRODUCT_ID GROUP BY B.BRAND_NAME ORDER BY TOTAL_QUANTITY DESC;

BRAND_NAME                    TOTAL_QUANTITY TOTAL_REVENUE
----------------------------- -------------- -------------
Gucci                                      3          3600
Armani                                     2          2800
Dior                                       2          3000
Chanel                                     1          1800
Versace                                    1          2200
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID HAVING SUM(TOTAL_AMOUNT)>(SELECT AVG(TOTAL_SPENDING) FROM
(SELECT SUM(TOTAL_AMOUNT) TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOME TOTAL_SPENDING
------- --------------
C-1003            3500
C-1005            4500
```

Query 3: Category Performance Analysis

Find:

- Highest revenue-generating category.

```
SQL> SELECT C.CATEGORY_NAME,SUM(OD.QUANTITY*B.PRICE) TOTAL_REVENUE FROM CATEGORY C JOIN BRAND B ON C.CATEGORY_ID=B.CATEGORY_ID JOIN ORDER_DETAILS OD
 ON B.PRODUCT_ID=OD.PRODUCT_ID GROUP BY C.CATEGORY_NAME ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME                 TOTAL_REVENUE
----------------------------- -------------
Fresh                                  3600
Floral                                 3000
Citrus                                 2800
Oriental                               2200
Woody                                  1800
```

- Number of products sold in each category.

```
SQL> SELECT C.CATEGORY_NAME,SUM(OD.QUANTITY) TOTAL_SOLD FROM CATEGORY C JOIN BRAND B ON C.CATEGORY_ID=B.CATEGORY_ID JOIN ORDER_DETAILS OD ON B.PRODU
CT_ID=OD.PRODUCT_ID GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                 TOTAL_SOLD
----------------------------- ----------
Floral                                 2
Oriental                               1
Citrus                                 2
Woody                                  1
Fresh                                  3
```

- Average product price by category.

```
SQL> SELECT C.CATEGORY_NAME,AVG(B.PRICE) AVG_PRICE FROM CATEGORY C JOIN BRAND B ON C.CATEGORY_ID=B.CATEGORY_ID GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                  AVG_PRICE
------------------------------ ----------
Musky                                1900
Floral                               1500
Oriental                             2200
Aquatic                              2500
Citrus                               1400
Vanilla                              3500
Luxury                               2000
Woody                                1800
Fresh                                1200
Fruity                               1600
```

Query 4: Customer Purchase History

Display:

- Customer name
- Number of orders
- Total purchase amount
- Most purchased product

```
SQL> SELECT C.CUSTOMER_ID,C.FIRST_NAME||' '||C.LAST_NAME CUSTOMER_NAME,COUNT(O.ORDER_ID) TOTAL_ORDERS,SUM(O.TOTAL_AMOUNT) TOTAL_SPENDING FROM CUSTOM
ER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.FIRST_NAME,C.LAST_NAME;

CUSTOMER_I CUSTOMER_NAME                          TOTAL_ORDERS TOTAL_SPENDING
---------- ---------------------------------------- ------------ --------------
C-1001     Chiru Raju                                          1           2500
C-1002     Hema Bhaskar                                        1           1800
C-1005     Deepikaa Selvam                                     1           4500
C-1003     Seetha Mani                                         1           3500
C-1004     Abinaya Ramesh                                      1           1200
```

5. Prepare Advanced SQL Report

Report 1: Premium Product Report

Display:

- Product Name
- Category
- Price
- Stock Availability

Condition: Products above average

price.

```
SQL> SELECT BRAND_NAME,CATEGORY_ID,PRICE,STOCK_QUANTITY,CASE WHEN STOCK_QUANTITY>0 THEN 'AVAILABLE' ELSE 'OUT OF STOCK' END STOCK_STATUS FROM BRAND
WHERE PRICE>(SELECT AVG(PRICE) FROM BRAND);

BRAND_NAME                     CATEGORY_I      PRICE STOCK_QUANTITY STOCK_STATUS
------------------------------ ---------- ---------- -------------- ------------
Versace                        CAT-004          2200             15 AVAILABLE
Calvin                         CAT-007          3500             10 AVAILABLE
YSL                            CAT-009          2500             18 AVAILABLE
Fogg                           CAT-010          2000             24 AVAILABLE
```

Report 2: Customer Value Report

Display:

- Customer Name
- Total Orders
- Total Spending

```
SQL> SELECT C.CUSTOMER_ID,C.FIRST_NAME||' '||C.LAST_NAME CUSTOMER_NAME,COUNT(O.ORDER_ID) TOTAL_ORDERS,SUM(O.TOTAL_AMOUNT) TOTAL_SPENDING,CASE WHEN S
UM(O.TOTAL_AMOUNT)>=100000 THEN 'PREMIUM' ELSE 'REGULAR' END CUSTOMER_CATEGORY FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY
 C.CUSTOMER_ID,C.FIRST_NAME,C.LAST_NAME;

CUSTOMER_I CUSTOMER_NAME                           TOTAL_ORDERS TOTAL_SPENDING
---------- --------------------------------------- ------------ --------------
CUSTOME
-------
C-1001     Chiru Raju                                         1           2500
REGULAR

C-1002     Hema Bhaskar                                       1           1800
REGULAR

C-1005     Deepikaa Selvam                                    1           4500
REGULAR


CUSTOMER_I CUSTOMER_NAME                           TOTAL_ORDERS TOTAL_SPENDING
---------- --------------------------------------- ------------ --------------
CUSTOME
-------
C-1003     Seetha Mani                                        1           3500
REGULAR

C-1004     Abinaya Ramesh                                     1           1200
REGULAR
```

Report 3: Sales Performance Report

Display:

```
SQL> SELECT B.BRAND_NAME,SUM(OD.QUANTITY) TOTAL_QUANTITY,SUM(OD.QUANTITY*B.PRICE) TOTAL_REVENUE FROM BRAND B JOIN ORDER_DETAILS OD ON B.PRODUCT_ID=O
D.PRODUCT_ID GROUP BY B.BRAND_NAME;

BRAND_NAME                    TOTAL_QUANTITY TOTAL_REVENUE
----------------------------- -------------- -------------
Chanel                                     1          1800
Armani                                     2          2800
Dior                                       2          3000
Gucci                                      3          3600
Versace                                    1          2200
```

- Product-wise sales
- Category-wise revenue

```
SQL> SELECT C.CATEGORY_NAME,SUM(OD.QUANTITY*B.PRICE) TOTAL_REVENUE FROM CATEGORY C JOIN BRAND B ON C.CATEGORY_ID=B.CATEGORY_ID JOIN ORDER_DETAILS OD
 ON B.PRODUCT_ID=OD.PRODUCT_ID GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                 TOTAL_REVENUE
----------------------------- -------------
Floral                                3000
Oriental                              2200
Citrus                                2800
Woody                                 1800
Fresh                                 3600
```

- Top-selling products

```
SQL> SELECT * FROM (SELECT B.BRAND_NAME,SUM(OD.QUANTITY) TOTAL_SOLD FROM BRAND B JOIN ORDER_DETAILS OD ON B.PRODUCT_ID=OD.PRODUCT_ID GROUP BY B.BRAN
D_NAME ORDER BY TOTAL_SOLD DESC) WHERE ROWNUM<=5;

BRAND_NAME                    TOTAL_SOLD
----------------------------- ----------
Gucci                                  3
Armani                                 2
Dior                                   2
Chanel                                 1
Versace                                1
```

Report 4: Business Decision Report

Identify:

- High-performing products

```
SQL> SELECT B.BRAND_NAME,SUM(OD.QUANTITY) TOTAL_SOLD FROM BRAND B JOIN ORDER_DETAILS OD ON B.PRODUCT_ID=OD.PRODUCT_ID GROUP BY B.BRAND_NAME HAVING S
UM(OD.QUANTITY)>(SELECT AVG(TOTAL_SOLD) FROM (SELECT SUM(QUANTITY) TOTAL_SOLD FROM ORDER_DETAILS GROUP BY PRODUCT_ID));

BRAND_NAME                    TOTAL_SOLD
------------------------------ ----------
Armani                                 2
Dior                                   2
Gucci                                  3
```

- Low-performing products

```
SQL> SELECT B.BRAND_NAME,SUM(OD.QUANTITY) TOTAL_SOLD FROM BRAND B JOIN ORDER_DETAILS OD ON B.PRODUCT_ID=OD.PRODUCT_ID GROUP BY B.BRAND_NAME HAVING S
UM(OD.QUANTITY)<(SELECT AVG(TOTAL_SOLD) FROM (SELECT SUM(QUANTITY) TOTAL_SOLD FROM ORDER_DETAILS GROUP BY PRODUCT_ID));

BRAND_NAME                    TOTAL_SOLD
------------------------------ ----------
Chanel                                 1
Versace                                1
```

- High-value customers

```
SQL> SELECT CUSTOMER_ID,TOTAL_SPENDING FROM (SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID) WHERE TOTAL_SPEND
ING=(SELECT MAX(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOME TOTAL_SPENDING
------- --------------
C-1005            4500
```

- Revenue trends

```
SQL> SELECT TO_CHAR(ORDER_DATE,'MON-YYYY') AS MONTH, SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS GROUP BY TO_CHAR(ORDER_DATE,'MON-YYYY'), TO_CHAR
(ORDER_DATE,'YYYYMM') ORDER BY TO_CHAR(ORDER_DATE,'YYYYMM');

MONTH             TOTAL_REVENUE
----------------- -------------
SEP-2026                  13500
```